

Batch effects and confounders

Jeff Leek

- Dependencies
- Download the data
- Set up the data
- [Adjusting for batch effects with a linear model](#)
- Adjusting for batch effects with Combat
- Comparing Combat and linear adjustment
- Adjusting for batch effects with sva
- Principal components for population structure
- Notes and further resources
- Session information

Dependencies

This document depends on the following packages:

```
library(devtools)
library(Biobase)
library(sva)
library(bladderbatch)
library(snpStats)
```

To install these packages you can use the code (or if you are compiling the document, remove the `eval=FALSE` from the chunk.)

```
install.packages(c("devtools"))
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Biobase", "sva", "bladderbatch", "snpStats"))
```

Download the data

The analyses performed in this experiment are based on gene expression measurements from a bladder cancer study: Gene expression in the urinary bladder: a common carcinoma in situ gene expression signature exists disregarding histopathological classification. (<http://cancerres.aacrjournals.org/content/64/11/4040.full.pdf>) The data can be loaded from the bladderbatch (<http://bioconductor.org/packages/release/data/experiment/html/bladderbatch.html>) data package.

```
data(bladderdata)
```

Set up the data

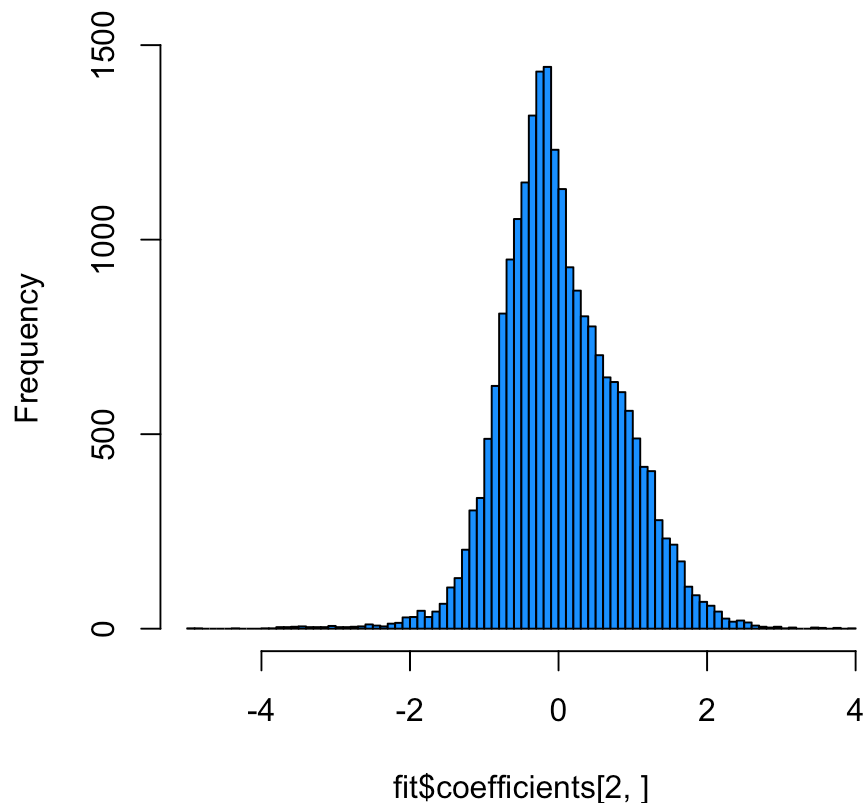
```
pheno = pData(bladderEset)
edata = exprs(bladderEset)
```

Adjusting for batch effects with a linear model

We will use two models. One with the variable we care about (cancer status) and the other that is just the known adjustment variables (in this case we will assume none)

```
mod = model.matrix(~as.factor(cancer) + as.factor(batch),data=pheno)
fit = lm.fit(mod,t(edata))
hist(fit$coefficients[2,],col=2,breaks=100)
```

Histogram of fit\$coefficients[2,]



This will only work if the batch effects aren't too highly correlated with the outcome. Let's check that

```
table(pheno$cancer,pheno$batch)
```

```
##
##           1  2  3  4  5
## Biopsy   0  0  0  5  4
## Cancer  11 14  0  0 15
## Normal   0  4  4  0  0
```

Adjusting for batch effects with Combat

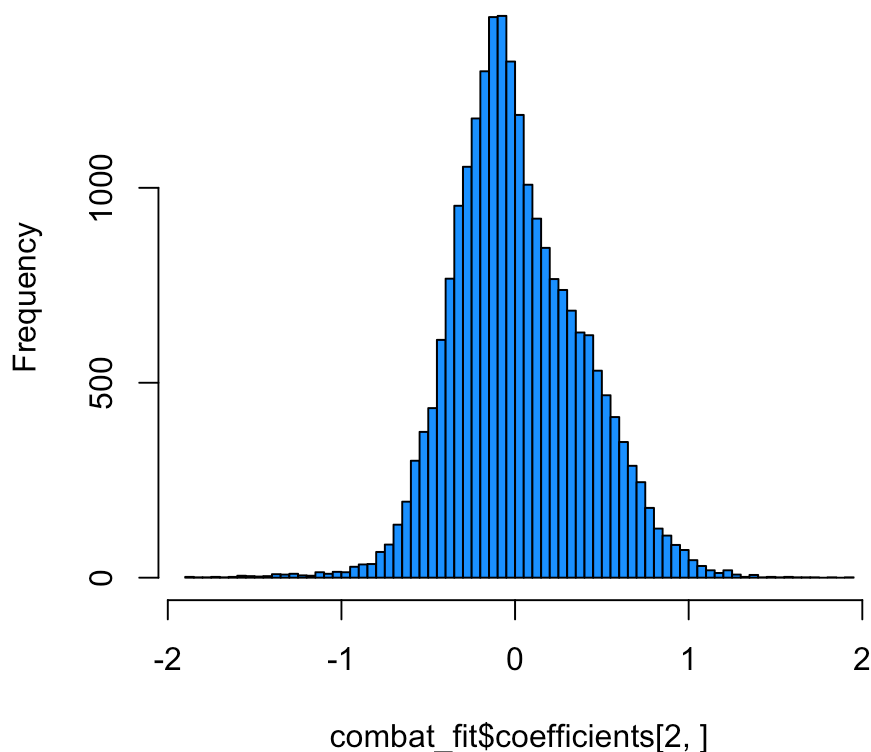
Another approach is to use Combat. Combat returns a “cleaned” data matrix after batch effects have been removed. Here we pass a model matrix with any known adjustment variables and a second parameter that is the batch variable.

```
batch = pheno$batch
modcombat = model.matrix(~1, data=pheno)
modcancer = model.matrix(~cancer, data=pheno)
combat_edata = ComBat(dat=edata, batch=batch, mod=modcombat, par.prior=TRUE, prior.plots=FALSE)
```

```
## Found 5 batches
## Adjusting for 0 covariate(s) or covariate level(s)
## Standardizing Data across genes
## Fitting L/S model and finding priors
## Finding parametric adjustments
## Adjusting the Data
```

```
combat_fit = lm.fit(modcancer,t(combat_edata))
hist(combat_fit$coefficients[2,],col=2,breaks=100)
```

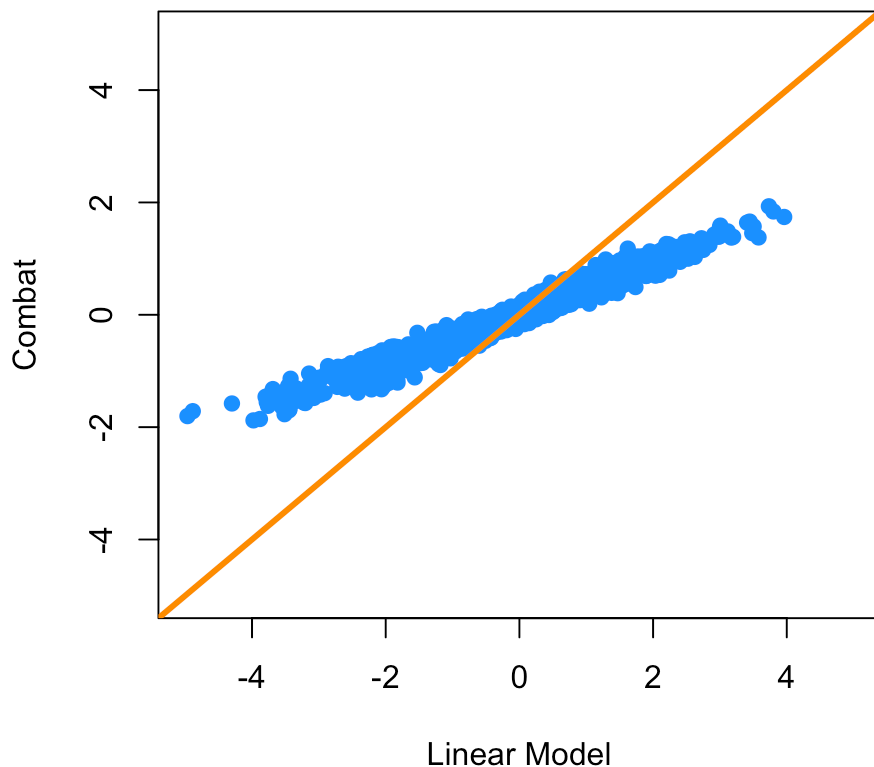
Histogram of combat_fit\$coefficients[2,]



Comparing Combat and linear adjustment

We can compare the estimated coefficients from Combat and linear adjustment by looking at the right coefficients for each model.

```
plot(fit$coefficients[2,],combat_fit$coefficients[2,],col=2,  
     xlab="Linear Model",ylab="Combat",xlim=c(-5,5),ylim=c(-5,5))  
abline(c(0,1),col=1,lwd=3)
```



Adjusting for batch effects with sva

First we need to estimate the surrogate variables. To do this, we need to build a model with any known adjustment variables and the variable we care about `mod` and another model with only the adjustment variables. Here we won't adjust for anything to see if `sva` can “discover” the batch effect.

```
mod = model.matrix(~cancer,data=pheno)  
mod0 = model.matrix(~1, data=pheno)  
sva1 = sva(edata,mod,mod0,n.sv=2)
```

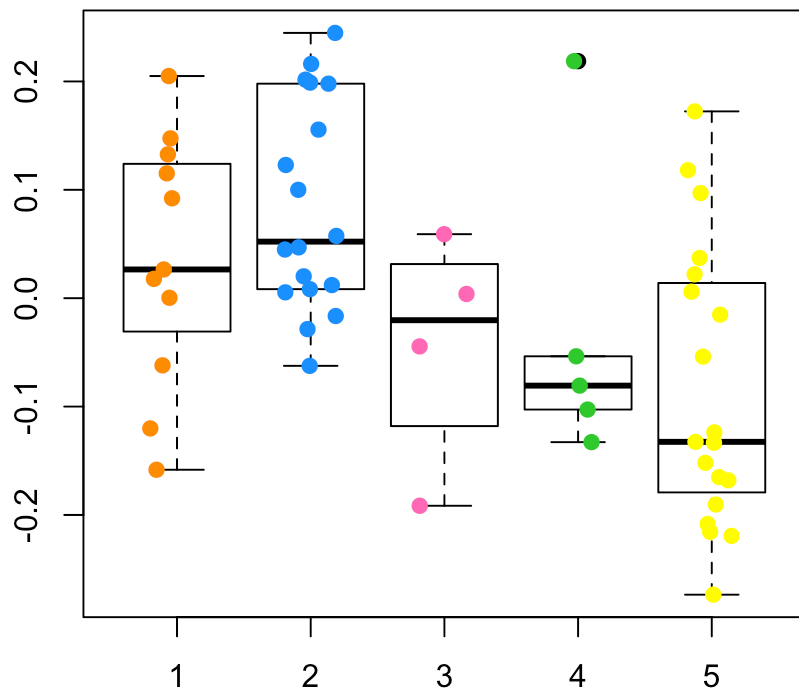
```
## Number of significant surrogate variables is: 2  
## Iteration (out of 5 ):1 2 3 4 5
```

See if any of the variables correlate with batch

```
summary(lm(sva1$sv ~ pheno$batch))
```

```
## Response Y1 :
##
## Call:
## lm(formula = Y1 ~ pheno$batch)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.26953 -0.11076  0.00787  0.10399  0.19069
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.018470   0.038694  -0.477   0.635
## pheno$batch  0.006051   0.011253   0.538   0.593
##
## Residual standard error: 0.1345 on 55 degrees of freedom
## Multiple R-squared:  0.00523,    Adjusted R-squared:  -0.01286
## F-statistic: 0.2891 on 1 and 55 DF,  p-value: 0.5929
##
##
## Response Y2 :
##
## Call:
## lm(formula = Y2 ~ pheno$batch)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.23973 -0.07467 -0.02157  0.08116  0.25629
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.121112   0.034157   3.546 0.000808 ***
## pheno$batch -0.039675   0.009933  -3.994 0.000194 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1187 on 55 degrees of freedom
## Multiple R-squared:  0.2248, Adjusted R-squared:  0.2107
## F-statistic: 15.95 on 1 and 55 DF,  p-value: 0.0001945
```

```
boxplot(sva1$sv[,2] ~ pheno$batch)
points(sva1$sv[,2] ~ jitter(as.numeric(pheno$batch)),col=as.numeric(pheno$batch))
```

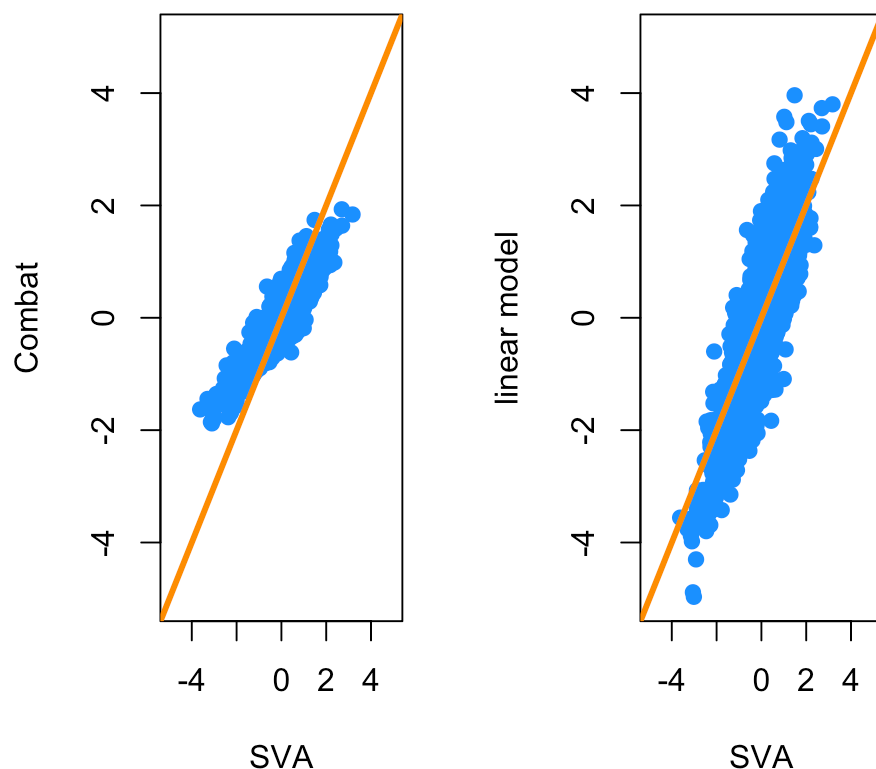


Add the surrogate variables to the model matrix and perform the model fit

```
modsv = cbind(mod, sva1$sv)
fitsv = lm.fit(modsv, t(edata))
```

Compare the fit from surrogate variable analysis to the other two.

```
par(mfrow=c(1,2))
plot(fitsv$coefficients[2,], combat_fit$coefficients[2,], col=2,
     xlab="SVA", ylab="Combat", xlim=c(-5,5), ylim=c(-5,5))
abline(c(0,1), col=1, lwd=3)
plot(fitsv$coefficients[2,], fit$coefficients[2,], col=2,
     xlab="SVA", ylab="linear model", xlim=c(-5,5), ylim=c(-5,5))
abline(c(0,1), col=1, lwd=3)
```



Principal components for population structure

Load an example data set and take a smaller subset of samples for computational efficiency

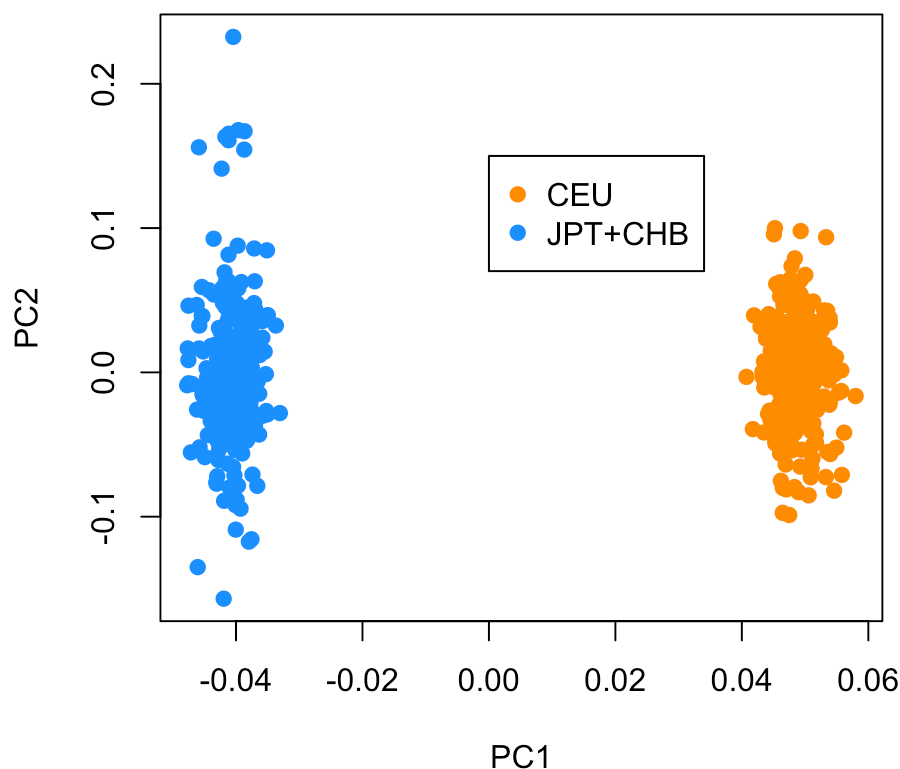
```
data(for.exercise)
controls <- rownames(subject.support)[subject.support$cc==0]
use <- seq(1, ncol(snps.10), 10)
ctl.10 <- snps.10[controls,use]
```

Calculate the PCs

```
xxmat <- xxt(ctl.10, correct.for.missing=FALSE)
evv <- eigen(xxmat, symmetric=TRUE)
pcs <- evv$vectors[,1:5]
```

Let's compare the PCs to the population labels and see that PC1 captures the population variable very well

```
pop <- subject.support[controls,"stratum"]
plot(pcs[,1],pcs[,2],col=as.numeric(pop),
     xlab="PC1",ylab="PC2")
legend(0,0.15,legend=levels(pop),pch=19,col=1:2)
```



Notes and further resources

You can read more about batch effect corrections in the sva package vignette (<https://www.bioconductor.org/packages/release/bioc/html/sva.html>). There are actually a large number of variants on the sva batch correction idea. Some good ones are:

- PEER (<https://www.sanger.ac.uk/resources/software/peer/>)
- RUV (<http://www.bioconductor.org/packages/release/bioc/html/RUVnormalize.html>)

There are also some alternatives for PCA analysis in R in addition to the usual `svd` and `pr.comp` functions.

- [SNPRelate][<http://www.bioconductor.org/packages/release/bioc/html/SNPRelate.html>]
(<http://www.bioconductor.org/packages/release/bioc/html/SNPRelate.html>)

Session information

Here is the session information

```
devtools::session_info()
```



```

## setting value
## version R version 3.2.1 (2015-06-18)
## system x86_64, darwin10.8.0
## ui RStudio (0.99.447)
## language (EN)
## collate en_US.UTF-8
## tz America/New_York
##
## package * version date
## acepack 1.3-3.3 2014-11-24
## annotate 1.46.1 2015-07-11
## AnnotationDbi * 1.30.1 2015-04-26
## assertthat 0.1 2013-12-06
## BiasedUrn * 1.06.1 2013-12-29
## Biobase * 2.28.0 2015-04-17
## BiocGenerics * 0.14.0 2015-04-17
## BiocInstaller * 1.18.4 2015-07-22
## BiocParallel 1.2.20 2015-08-07
## biomaRt 2.24.0 2015-04-17
## Biostrings 2.36.3 2015-08-12
## bitops 1.0-6 2013-08-17
## bladderbatch * 1.6.0 2015-08-26
## broom * 0.3.7 2015-05-06
## caTools 1.17.1 2014-09-10
## cluster 2.0.3 2015-07-21
## colorspace 1.2-6 2015-03-11
## corpcor 1.6.8 2015-07-08
## curl 0.9.2 2015-08-08
## DBI * 0.3.1 2014-09-24
## dendextend * 1.1.0 2015-07-31
## DESeq2 * 1.8.1 2015-05-02
## devtools * 1.8.0 2015-05-09
## digest 0.6.8 2014-12-31
## dplyr * 0.4.3 2015-09-01
## edge * 2.1.0 2015-09-06
## evaluate 0.7.2 2015-08-13
## foreign 0.8-66 2015-08-19
## formatR 1.2 2015-04-21
## Formula * 1.2-1 2015-04-07
## futile.logger 1.4.1 2015-04-20
## futile.options 1.0.0 2010-04-06
## gdata 2.17.0 2015-07-04
## genefilter * 1.50.0 2015-04-17
## geneLenDataBase * 1.4.0 2015-09-06
## geneplotter 1.46.0 2015-04-17
## GenomeInfoDb * 1.4.2 2015-08-15
## GenomicAlignments 1.4.1 2015-04-24
## GenomicFeatures 1.20.2 2015-08-14
## GenomicRanges * 1.20.5 2015-06-09
## genstats * 0.1.02 2015-09-05
## ggplot2 * 1.0.1 2015-03-17
## git2r 0.11.0 2015-08-12

```

##	GO.db	3.1.2	2015-09-06
##	goseq	* 1.20.0	2015-04-17
##	gplots	* 2.17.0	2015-05-02
##	gridExtra	2.0.0	2015-07-14
##	gtable	0.1.2	2012-12-05
##	gtools	3.5.0	2015-05-29
##	highr	0.5	2015-04-21
##	HistData	* 0.7-5	2014-04-26
##	Hmisc	* 3.16-0	2015-04-30
##	htmltools	0.2.6	2014-09-08
##	httr	1.0.0	2015-06-25
##	IRanges	* 2.2.7	2015-08-09
##	KernSmooth	2.23-15	2015-06-29
##	knitr	* 1.11	2015-08-14
##	lambda.r	1.1.7	2015-03-20
##	lattice	* 0.20-33	2015-07-14
##	latticeExtra	0.6-26	2013-08-15
##	lazyeval	0.1.10	2015-01-02
##	limma	* 3.24.15	2015-08-06
##	lme4	1.1-9	2015-08-20
##	locfit	1.5-9.1	2013-04-20
##	magrittr	1.5	2014-11-22
##	MASS	* 7.3-43	2015-07-16
##	Matrix	* 1.2-2	2015-07-08
##	MatrixEQTL	* 2.1.1	2015-02-03
##	memoise	0.2.1	2014-04-22
##	mgcv	* 1.8-7	2015-07-23
##	minqa	1.2.4	2014-10-09
##	mnormt	1.5-3	2015-05-25
##	munsell	0.4.2	2013-07-11
##	nlme	* 3.1-122	2015-08-19
##	nloptr	1.0.4	2014-08-04
##	nnet	7.3-10	2015-06-29
##	org.Hs.eg.db	* 3.1.2	2015-07-17
##	plyr	1.8.3	2015-06-12
##	preprocessCore	* 1.30.0	2015-04-17
##	proto	0.3-10	2012-12-22
##	psych	1.5.6	2015-07-08
##	qvalue	* 2.0.0	2015-04-17
##	R6	2.1.1	2015-08-19
##	RColorBrewer	1.1-2	2014-12-07
##	Rcpp	* 0.12.0	2015-07-25
##	RcppArmadillo	* 0.5.400.2.0	2015-08-17
##	RCurl	1.95-4.7	2015-06-30
##	reshape2	1.4.1	2014-12-06
##	rmarkdown	0.7	2015-06-13
##	rpart	4.1-10	2015-06-29
##	Rsamtools	1.20.4	2015-06-01
##	RSkittleBrewer	* 1.1	2015-09-05
##	RSQLite	* 1.0.0	2014-10-25
##	rstudioapi	0.3.1	2015-04-07
##	rtracklayer	1.28.9	2015-08-19

```

## rversions          1.0.2      2015-07-13
## S4Vectors          * 0.6.5      2015-09-01
## scales             0.3.0      2015-08-25
## snm                1.16.0     2015-04-17
## snpStats           * 1.18.0     2015-04-17
## stringi            0.5-5      2015-06-29
## stringr            1.0.0      2015-04-30
## survival           * 2.38-3     2015-07-02
## sva                * 3.14.0     2015-04-17
## tidyr              0.2.0      2014-12-05
## UsingR             * 2.0-5      2015-08-06
## whisker            0.3-2      2013-04-28
## XML                3.98-1.3    2015-06-30
## xml2               0.1.2      2015-09-01
## xtable             1.7-4      2014-09-12
## XVector            0.8.0      2015-04-17
## yaml               2.1.13     2014-06-12
## zlibbioc           1.14.0     2015-04-17
## source
## CRAN (R 3.2.0)
## Bioconductor
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## CRAN (R 3.2.0)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.2)
## Github (jdstorey/edge@a1947b5)
## CRAN (R 3.2.2)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)

```

```
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## Bioconductor
## local
## CRAN (R 3.2.0)
## CRAN (R 3.2.2)
## Bioconductor
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
```

```
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.1)
## Bioconductor
## Github (alyssafranze/RSkittleBrewer@0a96a20)
## CRAN (R 3.2.0)
## CRAN (R 3.2.0)
## Bioconductor
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.2)
## Bioconductor
## Bioconductor
## CRAN (R 3.2.1)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## Bioconductor
## CRAN (R 3.2.0)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## CRAN (R 3.2.1)
## CRAN (R 3.2.2)
## CRAN (R 3.2.0)
## Bioconductor
## CRAN (R 3.2.0)
## Bioconductor
```

It is also useful to compile the time the document was processed. This document was processed on: 2015-09-06.